AI-Powered B2B Applications

Contents

Overview 3

1. Dashboard 3

2. Admin 4

3. Notifications 4

4. Generated Messages 5

Business use case 6

Architecture 6

Frontend 6

Backend 6

Database design 7

AI feature 8

What the AI does 8

Why this is valuable 8

When messages are generated 8

What I would improve with more time 9

1. Better notification lifecycle 9
2. Smarter behavioral similarity 9
3. Better campaign intelligence 9
4. Multi-channel sending 9
5. Authentication and role-based access 9
6. Improve the design 9
7. Add interest scoring model instead manual interest score 9

Conclusion 10

Overview

This small AI-powered B2B CRM application (internal tool) helps marketing or sales teams identify high-interest customers, suggest relevant campaigns, and generate personalized outreach messages.

The core idea is simple: when a customer shows repeated strong interest through similar behaviors, the system flags that customer as a high-intent lead. From there, the user can review the signal, get an AI-based campaign suggestion, generate a personalized message, edit it if needed, and send it by email.

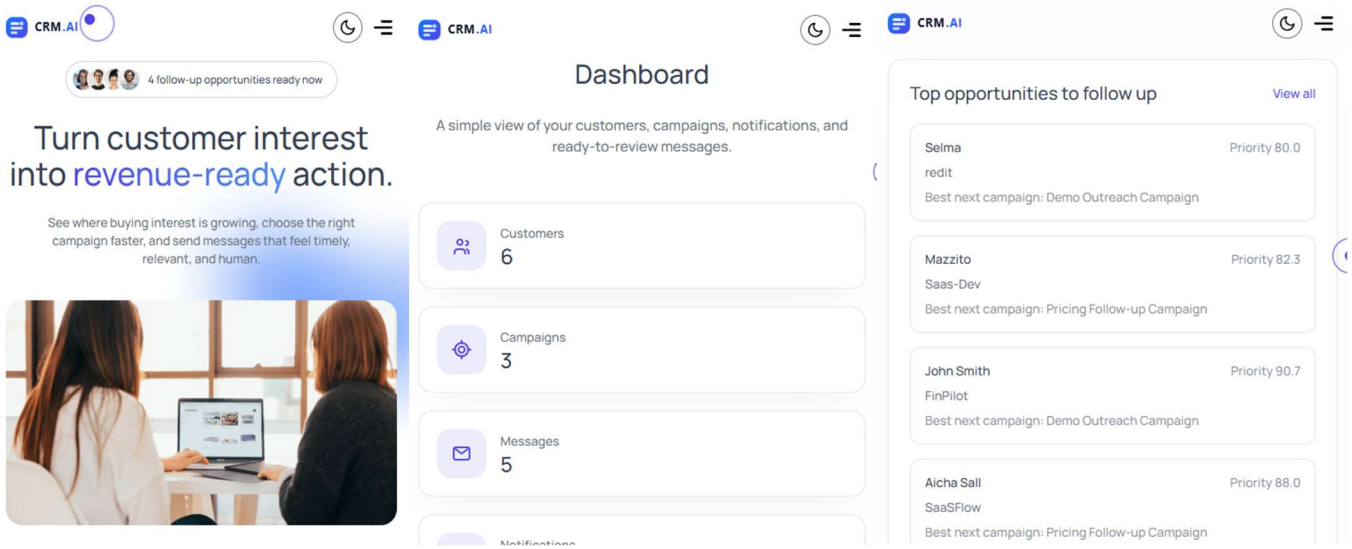
The application is a small full-stack CRM feature with four main areas:

1. Dashboard

The dashboard gives a quick overview of the CRM activity, including:

- total number of customers
- total number of campaigns
- total number of generated messages
- recent customer behaviors
- high-intent customers detected from repeated strong signals

This view is meant to simulate the daily operational dashboard used by a marketing or sales team.

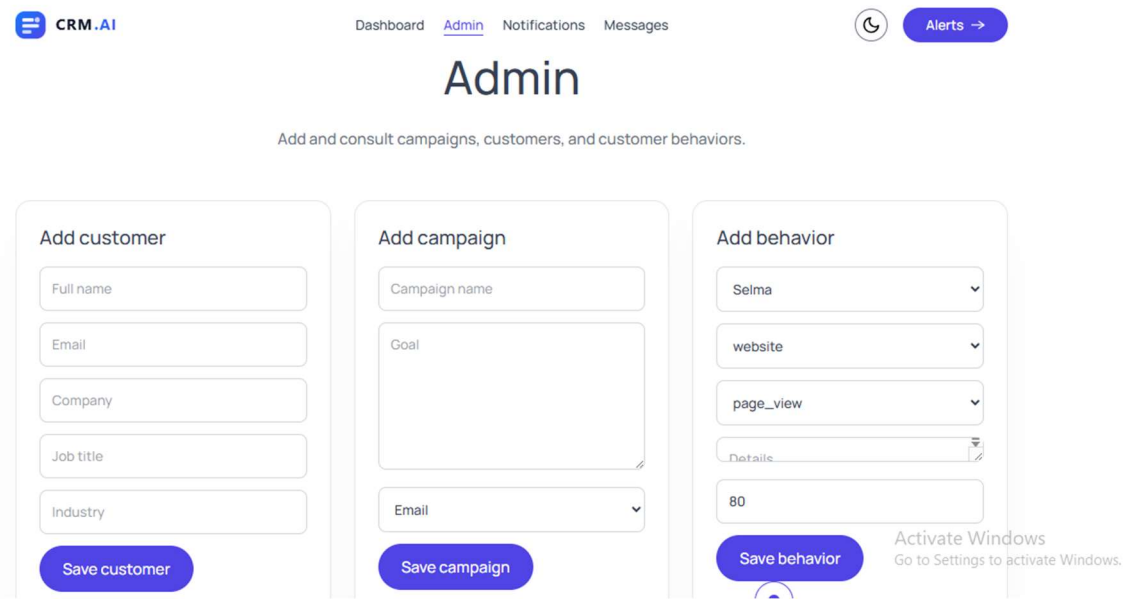


2. Admin

The admin area allows manual management of the main business entities:

- customers
- campaigns
- customer behaviors

This is useful for demo purposes because it allows inserting data manually without needing external integrations.



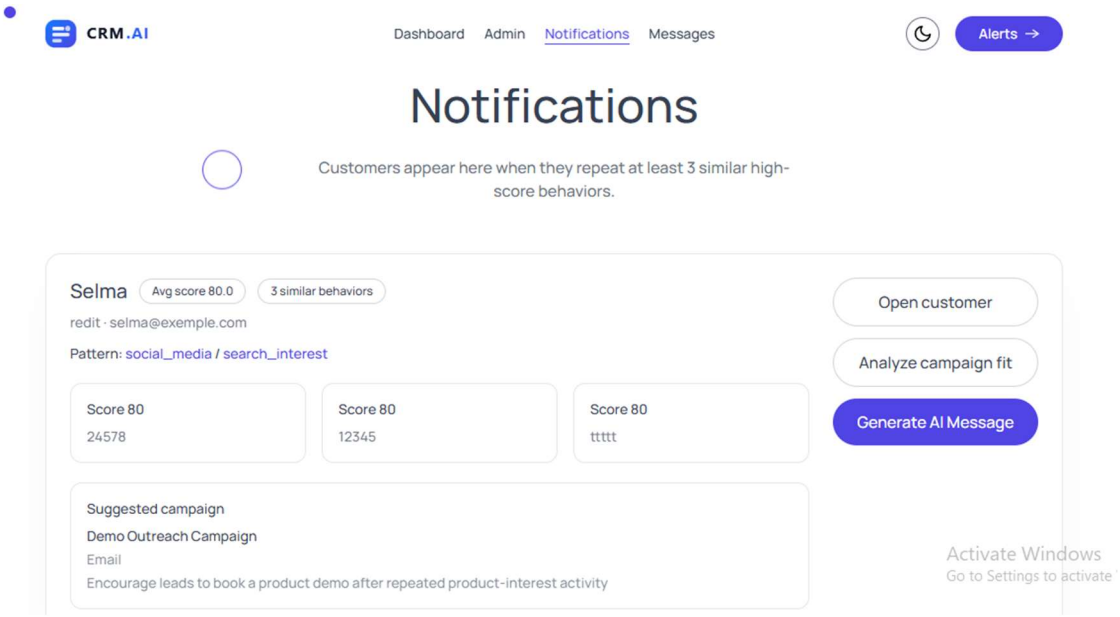
3. Notifications

The notifications page is where the main business logic becomes visible.

A customer appears in notifications when they have at least three similar behaviors with a high interest score. In the current logic, “similar” means the behaviors belong to the same customer and share the same source and behavior type, while each one has a score above the configured threshold.

From this page, the user can:

- identify high-intent customers
- review the repeated signals
- request an AI-based campaign suggestion
- trigger AI message generation



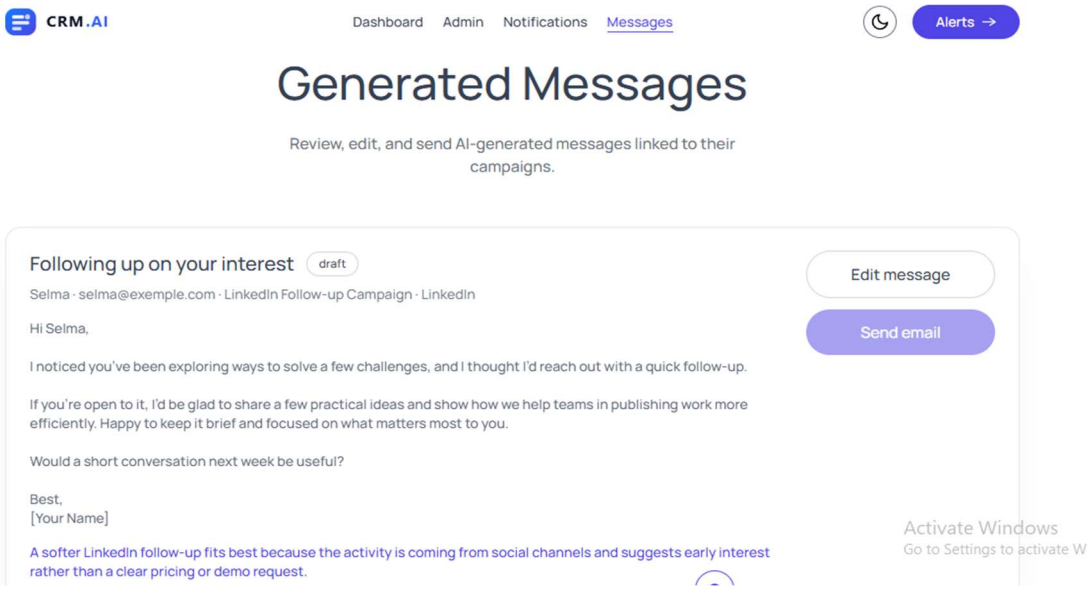
4. Generated Messages

Once a message is generated, it appears in the generated messages page.

From there, the user can:

- review the draft
- edit the subject and message body
- send the message through email

Only email sending is currently functional in the project (needs email added in the .env file). Other channels such as LinkedIn or SMS are represented in the campaign structure but are not implemented as active sending channels in this version.



Business use case

This application focuses on a B2B CRM and marketing use case.

In many companies, prospects interact with a product multiple times before converting. They may visit pricing pages, return to solution pages, repeatedly search for related topics, or engage with social content. Those actions often indicate real buying interest, but they are not always easy to detect and prioritize manually.

This tool helps solve that problem by:

- detecting repeated high-interest behavior
- surfacing the right customers at the right moment
- suggesting the most relevant campaign
- accelerating personalized outreach using AI

This makes the application useful for marketing, sales, or growth teams that want to respond faster and with more relevant communication.

Architecture

The project uses a full-stack architecture with a separate frontend and backend.

Frontend

The frontend is built in React and consumes the backend through REST API endpoints. It includes multiple views, form handling, loading states, and error messages.

The frontend was kept visually polished and modern to somehow match the expectation of a real B2B internal tool.

Backend

The backend is built with Flask and exposes a RESTful API. It handles:

- validation
- business logic
- PostgreSQL queries
- OpenAI integration
- email sending

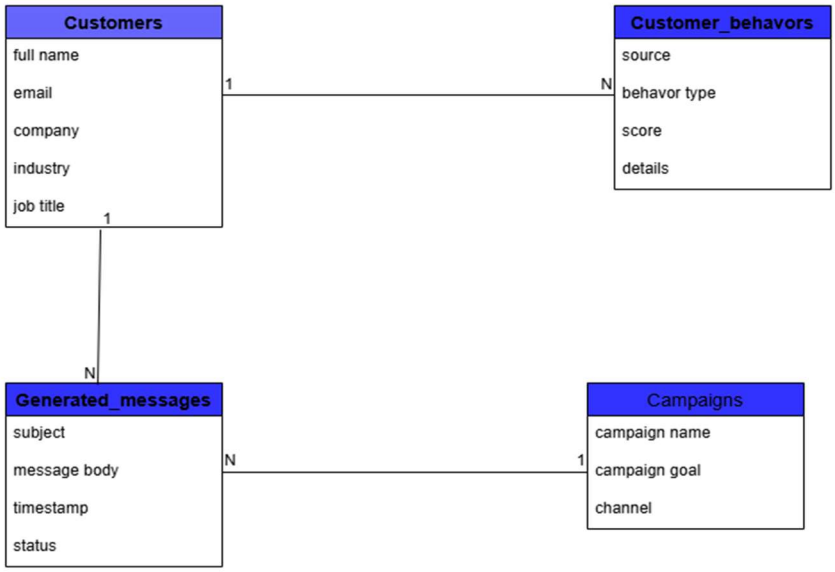
The backend was refactored into a modular structure to improve readability and maintainability.

Database design

The project uses a relational PostgreSQL database with related tables and seed data.

The main tables are:

customers	customer_behaviors	campaigns	generated_messages
• full name • email • company • job title • industry	• source • behavior type • description/details • score	• campaign name • campaign goal • channel	• subject • message body • status • timestamps



These tables are related in a way that supports the business workflow:

- one customer can have many behaviors
- one customer can receive many generated messages
- one campaign can be linked to many generated messages

AI feature

The main AI feature is embedded into the CRM workflow and adds direct business value.

What the AI does

When the user asks for analysis or message generation, the backend gathers:

- the customer profile
- the relevant customer behaviors
- the available campaigns

This information is sent to OpenAI so that the model can:

1. analyze the customer's likely interest
2. select the most appropriate campaign
3. generate a natural, marketing-oriented message

Why this is valuable

Instead of forcing the user to manually inspect repeated behaviors and decide what to send, the AI helps reduce that effort and produce a message draft faster. This makes the AI feature useful, contextual, and integrated into the real workflow.

When messages are generated

Messages are not generated automatically as soon as a behavior is added.

The workflow is intentionally designed in stages:

1. Customer behaviors are inserted into the database.
2. The system detects repeated similar high-interest behaviors.
3. The customer appears in the notifications page.
4. The user can request campaign analysis.
5. The user clicks "Generate AI Message".
6. The backend calls OpenAI and creates a draft message.
7. The user can edit the message or send it directly by email.

This design gives the user control and makes the AI feel like an assistant rather than an uncontrolled automation.

What I would improve with more time

Given more time, I would improve the project in the following directions:

1. Better notification lifecycle

Right now, high-intent detection is based on repeated behaviors. A stronger production version would store notifications as first-class records with statuses such as open, resolved, or dismissed.

2. Smarter behavioral similarity

The current demo defines similarity using source and behavior type. A more advanced version would support semantic similarity or richer grouping logic.

3. Better campaign intelligence

Campaign suggestion currently uses AI over the existing campaign list, but it could be improved with stronger campaign metadata, rules, and performance feedback loops.

4. Multi-channel sending

Only email is currently functional. A more complete version would support LinkedIn, SMS, or other outbound channels depending on business constraints.

5. Authentication and role-based access

This version focuses on the product workflow, but a real production version would include authentication, permissions, and user ownership.

6. Improve the design

The current interface is functional but minimal. A more polished version would include a refined UI/UX with better layout, consistent styling, improved responsiveness, and enhanced user interactions to provide a more intuitive and professional user experience.

7. Add interest scoring model instead manual interest score

Interest scores are currently assigned manually. A more advanced version would use a machine learning model or rule-based system to automatically calculate lead interest based on user behavior, engagement signals, and historical data, enabling more accurate and scalable decision-making.

Conclusion

This project is a small AI-powered CRM campaign copilot for B2B teams.

It detects high-interest customers from repeated behaviors, helps the user choose the right campaign, and generates personalized outreach messages using OpenAI.

The application combines:

- a React frontend
- a Flask REST API
- a PostgreSQL relational database
- a meaningful LLM-powered feature